

Fastest CLI

Geração Inteligente de Testes com IA

Pipeline Inteligente de Geração de Testes a partir de Requisitos em Linguagem Natural

Domínio SWEBOK: Software Testing (Capítulo 4) · v0.1.0

Equipe: Jorge Freitas (líder)

github.com/jorgelcff/Fastest-CLI

TypeScript · Node.js · Jest/Vitest · OpenAI · Anthropic

O Problema: Testes são Caros e Negligenciados

Dado	Valor
Tempo gasto em testes	15–30% do ciclo de desenvolvimento
Cobertura média em projetos reais	< 50%
Testes escritos após o código	> 70% dos projetos
Devs que consideram testes "chatos"	62% (Stack Overflow)

- ▶ Testes repetitivos consomem tempo criativo
- ▶ Qualidade inconsistente entre membros do time
- ▶ Cobertura cai conforme prazos apertam

Para Quem Construimos: Carlos e Marina



Carlos (Desenvolvedor Backend)

Foco: Fricção & Velocidade



Detesto escrever testes, mas sei que preciso para aprovar meu PR.

Perfil: 4 anos de experiência em Node.js/TypeScript.

Necessidade Operacional: Uma ferramenta que elimine o boilerplate e gere testes rapidamente a partir do código existente, alcançando 70% de cobertura com atrito zero.



Marina (Tech Lead)

Foco: Padronização & Escala



Cada desenvolvedor do meu time escreve testes de um jeito diferente.

Perfil: Gerencia um time de 6 desenvolvedores.

Necessidade Operacional: Uma ferramenta que garanta consistência arquitetural de testes, cubra edge cases automaticamente e reduza as 2h/semana gastas revisando testes em Code Reviews.

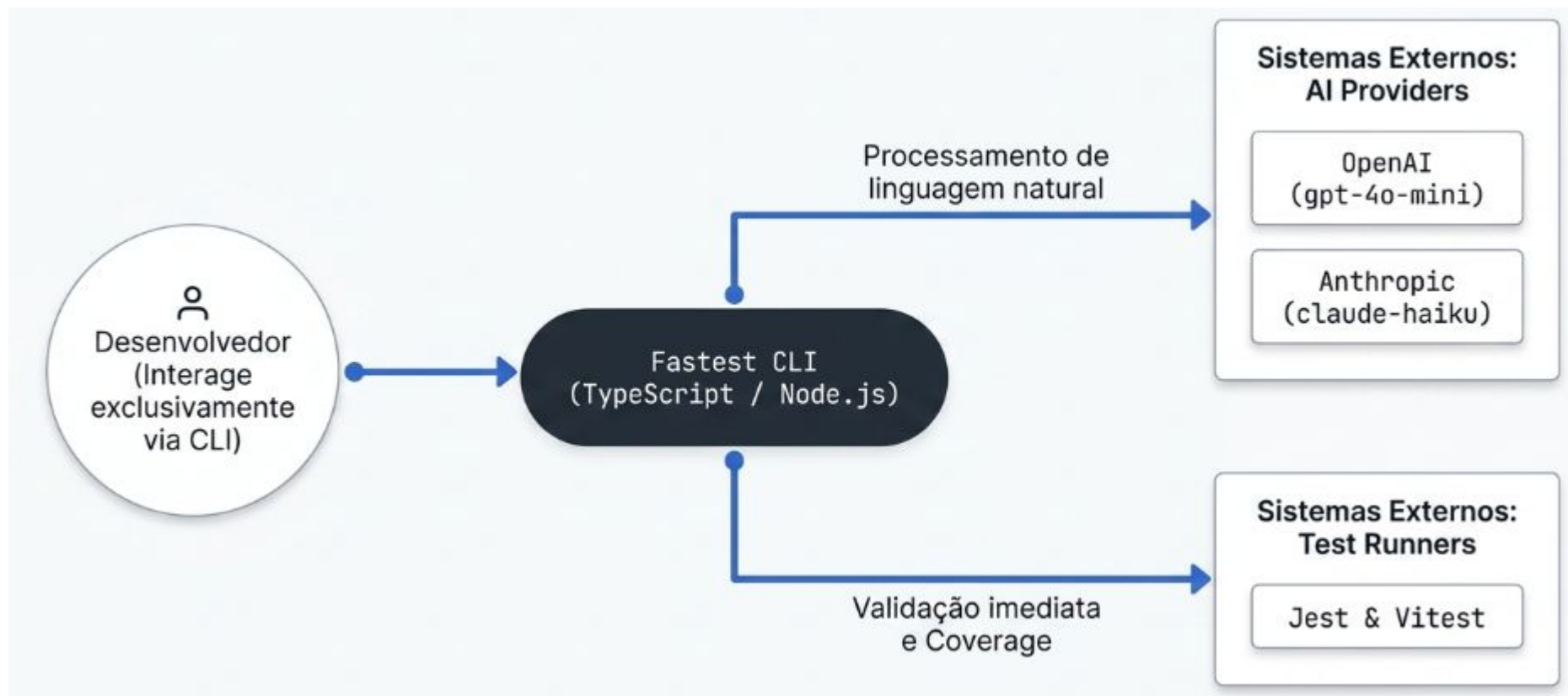
Missão, Visão e Indicadores-Chave

Missão: democratizar a escrita de testes de qualidade através de IA generativa, reduzindo a barreira de entrada e aumentando a cobertura.

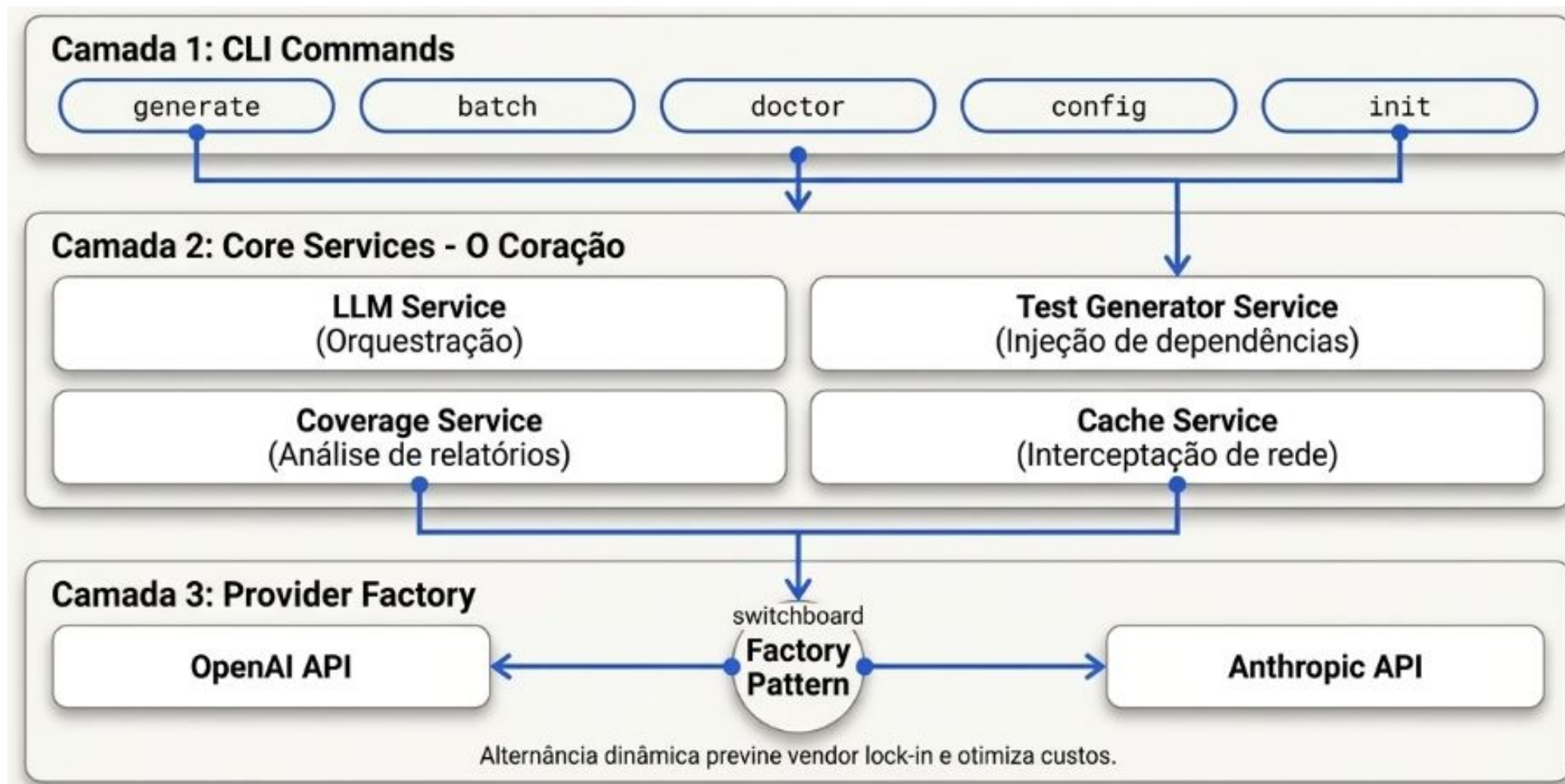
Visão: ser a ferramenta padrão de geração de testes em projetos TypeScript/JavaScript até 2027.

KPI	Meta	Status Atual
Cobertura do próprio projeto	≥ 90%	93%+
Compilação dos testes gerados	≥ 80%	Validado
Redução de tempo de escrita	≥ 50%	Validado
Número de testes automatizados	≥ 150	201
Taxa de passagem no Jest	≥ 60%	Validado

Arquitetura C4 — Diagrama de Contexto



Arquitetura C4 — Containers e Integração com IA



Decisões Arquiteturais e Trade-offs

Decisão	Alternativa Rejeitada	Justificativa
CLI (terminal)	App web / extensão IDE	Menor fricção, integra com CI/CD
Multi-provider (Factory)	Apenas OpenAI	Resiliência, evita vendor lock-in
Cache local em arquivo	Redis / banco	Simplicidade, zero infra
Temperature 0.2	Temperature alta	Testes determinísticos
Jest + Vitest	Apenas Jest	Vitest é mais rápido e cresce

► Trade-offs aceitos: dependência de API (cache+retry+dry-run), custo por chamada (modelos econômicos), código enviado a APIs (permissões+dry-run)

Catálogo de Prompts — 3 Prompts Centrais

1. buildTestPrompt (principal)

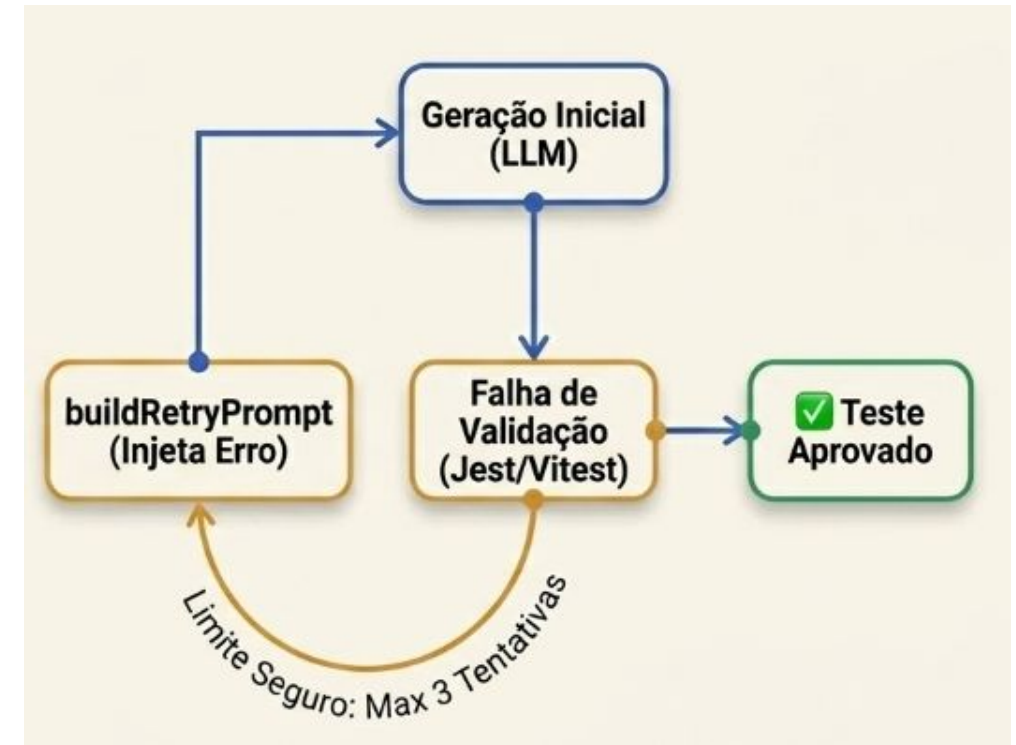
- ▶ Entrada: código-fonte, tipo (unit/integration/use-case), framework
- ▶ Saída: arquivo de teste completo · temperature 0.2, max_tokens 4096

2. buildRetryPrompt (correção)

- ▶ Entrada: teste gerado + erro de compilação/execução → teste corrigido
- ▶ Loop de até 3 tentativas com feedback do erro

3. buildCoverageSuggestionPrompt (otimização)

- ▶ Entrada: relatório de cobertura + código → sugestões de testes adicionais
- ▶ Prompts versionados no código (rastreabilidade e reproducibilidade)



Canvas de Experimento — Hipótese e GO/NO-GO

Hipótese: uma CLI alimentada por IA reduz o tempo de escrita de testes em 60%, mantendo qualidade de cobertura equivalente.

Critério GO	Meta	Resultado	Status
Compilação TypeScript	≥ 80%	Validado	GO
Taxa de passagem no Jest	≥ 60%	Validado	GO
Cobertura de código	≥ 70%	93%+	GO
Redução de tempo	≥ 50%	Validado	GO

Resultado: todos os critérios GO atendidos. Pipeline validada end-to-end.

DEMO

Estratégia de Testes — 201 Testes, 93%+ Cobertura

Marco	Testes	Cobertura
v0.0.1 (baseline)	154	81,68%
v0.0.2 (refatoração)	~180	~88%
v0.1.0 (atual)	201	93%+

Distribuição por módulo

- ▶ CLI Commands: generate, batch, doctor, config, init
- ▶ Services: llm, test-generator, coverage, cache (mocks de API)
- ▶ Providers: openai, anthropic, factory (respostas simuladas)
- ▶ Edge cases: inputs inválidos, timeouts, erros de API, cache miss/hit

Qualidade dos Testes Gerados pela IA

O que a IA faz bem

- ▶ Estrutura de teste (describe/it) consistente
- ▶ Identificação de happy paths e geração de mocks adequados
- ▶ Cobertura de branches básicas

Onde a IA precisa de ajuda

- ▶ Lógica de negócio complexa e específica do domínio
- ▶ Testes de integração com estado compartilhado
- ▶ Assertions de performance e edge cases muito específicos

Mecanismo de correção: retry com feedback do erro (até 3 tentativas) → eleva a taxa de passagem.

Segurança e Confiabilidade

Risco	Mitigação
Código-fonte enviado a APIs externas	Documentação clara, consentimento, dry-run
API keys expostas	Permissões 0o600, .env no .gitignore
Prompt injection via código-fonte	Validação de input, sanitização
Testes com código malicioso	Validação antes da escrita, dry-run
Custo descontrolado de API	Cache local, limites de tokens, modelos econômicos

► --dry-run visualiza sem executar · cache evita reproprocessamento · nenhum dado persistido em servidores externos

Evolução: v0.0.2 → v0.1.0

Feature	v0.0.2	v0.1.0
Providers	Apenas OpenAI	OpenAI + Anthropic
Frameworks	Apenas Jest	Jest + Vitest
Comandos	4	6 (+batch)
Testes / Cobertura	154 / 81,68%	201 / 93%+
Retry	Sem retry	Até 3 tentativas com feedback
Validação	Mínima	Input validation completa

► Refatorações: Factory Pattern, separação de responsabilidades, tipagem TypeScript, batch mode

Resultados — Métricas vs Metas

Métrica	Meta	Resultado	Delta
Testes automatizados	≥ 150	201	+34%
Cobertura de código	≥ 70%	93%+	+23pp
Compilação dos outputs	≥ 80%	Atingido	GO
Passagem no Jest	≥ 60%	Atingido	GO
Comandos funcionais	4	6	+50%
Providers suportados	1	2	+100%

Pipeline validada: requisito → prompt → LLM → teste → execução → cobertura. Todos os GO atendidos.

Painel de Feedback — Temas e Insights

Feedback positivo

- ▶ "Gerar testes pelo terminal é muito mais rápido que manualmente"
- ▶ "O modo batch é um game-changer para projetos grandes"
- ▶ "Suporte a Vitest faltava em outras ferramentas"

Oportunidades

- ▶ Mais linguagens além de TypeScript/JavaScript
- ▶ Integração com IDE · testes de integração com banco de dados

Insight: batch mode é a feature mais valorizada por tech leads (Marina); generate simples agrada devs individuais (Carlos); dry-run é mais usado que o esperado.

Decisão: PERSEVERAR

Opção	Critério	Resultado
PERSEVERAR	Todos os critérios GO atendidos	SIM
PIVOTAR	Hipótese invalidada, domínio válido	N/A
ABANDONAR	Domínio sem viabilidade	N/A

- ▶ 201 testes, 93%+ cobertura (meta: 150 / 70%)
 - ▶ Pipeline end-to-end funcional · hipótese validada
 - ▶ Arquitetura extensível (Factory permite novos providers)
- Próximos passos: publicar v0.1.0, coletar métricas reais, expandir versão beta**

Roadmap — v0.5 e v1.0

v0.5 (próximo trimestre)

- ▶ Mais modelos (GPT-4o, Claude Sonnet) · watch mode
- ▶ Retry com análise semântica · prompts customizados · relatório HTML

v1.0 (6 meses)

- ▶ Suporte a Python e Go · plugin system para frameworks
- ▶ Integração VS Code / JetBrains · análise de mutação · CI/CD

Visão de longo prazo: agente autônomo de qualidade que sugere testes proativamente em PRs e aprende com as correções do time.

Reflexão: Uso de IA no Desenvolvimento

Onde	Ferramenta	Para quê
Desenvolvimento do CLI	Claude (Anthropic)	Codificação, refatoração, testes
Geração de testes (produto)	gpt-4o-mini, Claude Haiku	Core feature da ferramenta
Documentação	Claude	Revisão e estruturação

► Limitações: foco em happy path · outputs exigem revisão humana · dependência de APIs externas

*Atribuição: a IA foi usada como assistente no desenvolvimento. Todos os outputs foram revisados e validados.
A IA é ferramenta, não substituto.*

Obrigado